

TECH CHALLENGE
FASE 2

RELATÓRIO TÉCNICO - TECH CHALLENGE FASE 2

OTIMIZAÇÃO DE ROTAS PARA DISTRIBUIÇÃO DE MEDICAMENTOS E INSUMOS

CURSO: PÓS-GRADUAÇÃO EM IA PARA DEVS (FIAP)

TURMA: 11IADT

EQUIPE: Grupo 17

NOME	RM
Douglas de Souza Santos	RM376076
Elisangela de Souza Alves	RM376784
Thiago Oliveira Teberga	RM376295

LINK DO REPOSITÓRIO GITHUB:

https://github.com/PosTech-FIAP-2026/IADT-Fase2-Tech-challenge

LINK PARA VÍDEO DE APRESENTAÇÃO:

[INSERIR LINK DO VÍDEO DA FASE 2]

ORGANIZAÇÃO DO PROJETO:

```
IADT-Fase2-Tech-challenge/
├── .github/workflows/                = CI (testes + build) e deploy no GCP
├── backend/
│   ├── app/
│   │   ├── api/auth.py               = Login e validação do token JWT
│   │   ├── api/routes_controller.py  = Controller REST (FastAPI)
│   │   ├── services/
│   │   │   ├── vrp.py                = Representação genética, fitness e horários
│   │   │   ├── genetic_algorithm.py  = Operadores, reparo e laço do AG
│   │   │   ├── heuristics.py         = Vizinho mais próximo / aleatória
│   │   │   ├── google_maps.py        = Geocoding API + Routes API
│   │   │   ├── distance.py           = Provedor de distâncias (fallback)
│   │   │   ├── llm.py                = OpenRouter, prompts e histórico
│   │   │   └── optimizer_service.py  = Pré-verificação e orquestração
│   │   ├── data/catalog.py           = Grupos de medicamentos e veículos
│   │   ├── data/cities.py            = Cenário padrão + pool de sorteio
│   │   ├── data/geo.py               = Estados e municípios (IBGE)
│   │   ├── schemas.py / config.py    = DTOs (Pydantic) e configuração
│   │   └── main.py                   = App FastAPI + CORS
│   ├── scripts/experiments.py        = Experimentos com o AG
│   ├── tests/                        = 96 testes automatizados (pytest)
│   └── Dockerfile / requirements.txt / .env.example
├── frontend/                         = Angular 20 + Angular Material + Leaflet
│   ├── src/app/components/           = Login, mapa, resultado, LLM, cadastro
│   └── firebase.json                 = Firebase Hosting
├── docs/                             = Este relatório e figuras
└── README.md
```

Sumário

1. Resumo 5
2. Introdução e definição do problema 5
2.1 Contextualização 5
2.2 Problema de negócio/saúde 6
2.3 Objetivo 6
2.4 Restrições consideradas 7
2.5 Hipóteses e limitações iniciais 8
3. Dados e origem das localizações 8
4. Arquitetura da solução 10
4.1 Visão geral 10
4.2 Fluxo de uma otimização 11
4.3 Backend e endpoints 12
4.4 Frontend 13
4.5 Segurança 13
4.6 Implantação na nuvem e CI/CD 14
5. Implementação do algoritmo genético 14
5.1 Código base e modificações 14
5.2 Representação genética 15
5.3 População inicial 16
5.4 Seleção 16
5.5 Crossover 16
5.6 Mutação 16
5.7 Operador de reparo 17
5.8 Função fitness 17
5.9 Critério de parada e reprodutibilidade 18
5.10 Parâmetros 18
6. Estratégias para lidar com as restrições 18
6.1 Restrições flexíveis (soft) e rígidas (hard) 18
6.2 Prioridades 19
6.3 Capacidade de carga 19
6.4 Autonomia e abastecimento 19
6.5 Janelas de horário 19
6.6 Grupos de medicamentos: contaminação e refrigeração 20
6.7 Frota heterogênea e múltiplos veículos 20
6.8 Origem e destino distintos 20
7. Integração com LLMs 20

7.1 Abordagem 20
7.2 Dois públicos: análise técnica e motoristas 21
7.3 Prompts 21
7.4 Contexto enviado à LLM 22
7.5 Histórico de execuções na análise técnica 22
7.6 Perguntas em linguagem natural 23
7.7 Avaliação da qualidade e fallback 23
8. Experimentos e resultados 25
8.1 Configuração dos experimentos 25
8.2 Resultados por configuração 26
8.3 Comparativo com outras abordagens 27
8.4 Critério de parada por estagnação e reprodutibilidade 28
8.5 Impacto do peso de prioridade 30
8.6 Operador de reparo 30
8.7 Composição da frota 31
8.8 Abastecimento em trechos longos 31
9. Visualizações e análise das rotas 32
10. Testes automatizados 34
11. Desafios enfrentados e soluções 35
12. Discussão crítica 36
13. Conclusão 37
14. Referências 38

1. Resumo

Este trabalho desenvolve um sistema de otimização de rotas para a distribuição de medicamentos e insumos hospitalares, tratado como um problema de roteamento de veículos (VRP) e resolvido com um algoritmo genético construído a partir do código base de caixeiro viajante (TSP) fornecido pela FIAP. O modelo considera frota heterogênea (moto, carro, carro elétrico e caminhões com e sem refrigeração), prioridade das entregas, capacidade, autonomia com paradas para abastecimento, janelas de horário de recebimento e regras sanitárias de transporte entre grupos de medicamentos, com cada restrição configurável como flexível (soft) ou rígida (hard). Um operador de reparo trata as regras de contaminação e refrigeração, e o algoritmo tem semente para reprodutibilidade e critério de parada por estagnação. As localizações são cidades reais, validadas na base de municípios do IBGE e geocodificadas pelo Google Maps, que também fornece as distâncias e tempos por estrada. O sistema é exposto por uma API em Python (FastAPI) com autenticação JWT, consumida por uma interface em Angular, e publicado no Google Cloud (Cloud Run e Firebase Hosting) com CI/CD pelo GitHub Actions. Uma LLM pré-treinada, via OpenRouter, gera uma análise técnica com sugestões de parâmetros, que compara as execuções anteriores com as mesmas cidades, e as instruções para os motoristas, além de responder perguntas em linguagem natural. Nos experimentos, o algoritmo genético reduziu o fitness médio em 91,1% (15 cidades) e 74,6% (25 cidades) em relação à heurística do vizinho mais próximo, principalmente pela redução dos atrasos nas janelas de horário, com todas as restrições rígidas atendidas.

2. Introdução e definição do problema

2.1 Contextualização

A logística hospitalar envolve o abastecimento contínuo de unidades de saúde e o atendimento domiciliar com medicamentos, vacinas, hemoderivados e insumos. Parte desses itens é sensível ao tempo, seja pela cadeia de frio, seja pela urgência clínica do paciente, e a falta de um item crítico em uma unidade pode interromper tratamentos. Ao mesmo tempo, a frota disponível é limitada e

heterogênea: motos, carros, carros elétricos e caminhões, com ou sem refrigeração, cada um com capacidade de carga e autonomia próprias.

Na fase 1, o hospital universitário investiu em modelos de machine learning para apoio ao diagnóstico. Nesta fase, o desafio passa a ser operacional: organizar a distribuição entre as unidades de forma eficiente, respeitando regras sanitárias de transporte, e transmitir às equipes de entrega instruções claras sobre o que fazer.

2.2 Problema de negócio/saúde

O problema clássico do caixeiro viajante (TSP) busca a menor rota que visita um conjunto de pontos uma única vez. Na distribuição hospitalar, ele é insuficiente: há vários veículos de tipos diferentes, entregas com prioridades diferentes, janelas de horário de recebimento e grupos de medicamentos que não podem viajar juntos por risco de contaminação, além de cargas que exigem refrigeração. O problema resultante é uma variante do problema de roteamento de veículos (VRP) com frota heterogênea, capacidade (CVRP), janelas de tempo (VRPTW), limite de autonomia e restrições de compatibilidade de carga. Ele é NP-difícil: o número de soluções cresce de forma fatorial com a quantidade de pontos, e a enumeração exaustiva se torna inviável a partir de poucas dezenas de entregas.

Além de calcular a rota, é necessário comunicá-la. Uma lista de índices e distâncias não é útil para um motorista, que precisa de instruções passo a passo (o que carregar, horários, onde abastecer), nem para a equipe que calibra o sistema, que precisa de um diagnóstico da execução e de sugestões de ajuste. Esse é o papel da LLM.

2.3 Objetivo

Desenvolver um sistema de otimização de rotas para distribuição de medicamentos e insumos utilizando algoritmos genéticos e LLMs. Os objetivos específicos são:

- Adaptar o código base de TSP da FIAP para o problema de roteamento com múltiplos veículos de tipos diferentes;
- Definir uma representação genética adequada para rotas de vários veículos;

- Implementar operadores especializados de seleção, crossover, mutação e reparo;
- Construir uma função fitness que considere distância, prioridade, capacidade, autonomia, janelas de horário e regras de transporte, com cada restrição configurável como flexível (soft) ou rígida (hard);
- Utilizar localizações e distâncias reais por estrada, obtidas pela API do Google Maps;
- Calcular os horários de chegada e as paradas para abastecimento de cada veículo;
- Comparar o algoritmo genético com outras abordagens de roteamento (vizinho mais próximo e solução aleatória);
- Realizar experimentos com diferentes configurações do algoritmo genético, incluindo semente e critério de parada;
- Visualizar as rotas otimizadas em um mapa interativo;
- Integrar uma LLM pré-treinada para gerar uma análise técnica e as instruções dos motoristas, e para responder perguntas em linguagem natural;
- Disponibilizar a solução por uma API REST autenticada e uma interface web, publicadas na nuvem com CI/CD;
- Validar as funcionalidades com testes automatizados.

2.4 Restrições consideradas

O enunciado exige restrições realistas além da distância. A tabela resume como cada uma foi modelada e o modo padrão; os detalhes estão na seção 6. Restrições marcadas como "sempre rígida" não podem ser flexibilizadas.

Restrição	Modelagem no sistema	Modo padrão
Prioridade das entregas	Cada ponto é critical (medicamento crítico) ou regular. Entregas críticas atendidas mais tarde aumentam o fitness; no modo hard, nenhuma crítica pode vir depois de uma regular no mesmo veículo.	Soft
Capacidade de carga	Demanda em kg por ponto; capacidade por tipo de veículo. O excesso é penalizado.	Hard
Autonomia	Distância máxima com um tanque. Com abastecimento, os veículos a combustão param na cidade ou no caminho ao atingir a reserva de 15%.	Hard
Autonomia do carro elétrico	Não recarrega na rota: o percurso inteiro precisa caber na autonomia.	Sempre rígida

Janelas de horário	Cada entrega tem início e fim de recebimento; chegar antes gera espera, depois gera atraso.	Soft
Contaminação	Grupos A e C não viajam juntos; o grupo B viaja sozinho.	Sempre rígida
Refrigeração	O grupo B só pode ir em caminhão com refrigeração.	Sempre rígida
Frota heterogênea	Moto, carro, carro elétrico, caminhão com e sem refrigeração; um veículo pode ficar sem uso.	—
Origem e destino	Rotas saem da origem e terminam no destino escolhido; se iguais, ida e volta.	—
Distâncias reais	Matriz de distâncias e tempos por estrada obtida da Routes API do Google Maps.	—

2.5 Hipóteses e limitações iniciais

H1: o algoritmo genético encontra rotas melhores que heurísticas construtivas simples, como o vizinho mais próximo, especialmente quando o número de pontos e de restrições cresce.

H2: a inclusão de um termo de prioridade no fitness antecipa as entregas críticas com um aumento pequeno na distância total.

H3: tratar as restrições por penalidade, com pesos diferentes para soft e hard, permite explorar soluções inviáveis durante a busca e convergir para soluções viáveis quando elas existem.

H4: um operador de reparo específico para as regras sanitárias (contaminação e refrigeração) aumenta a proporção de soluções viáveis em relação a confiar só na penalidade.

H5: o critério de parada por estagnação reduz o tempo de execução sem perda relevante de qualidade.

Como limitações, o modelo não considera trânsito em tempo real nem mais de uma viagem por veículo no mesmo dia, e o tempo de descarga é o mesmo em todas as entregas. As demandas, prioridades, grupos e janelas das cidades sorteadas são gerados aleatoriamente, pois não há dados reais de pedidos do hospital.

3. Dados e origem das localizações

Diferentemente da fase 1, este projeto não parte de um dataset estático. Os dados do problema são as localizações das unidades de saúde, as demandas e

regras de cada uma e as distâncias entre elas, obtidas em tempo de execução a partir de serviços externos.

Fonte	Uso no sistema
Google Geocoding API	Converte o nome da cidade em latitude e longitude; no sentido inverso (reverse geocoding), dá o nome do município onde fica uma parada para abastecimento no caminho.
Google Routes API — computeRouteMatrix	Distância (m) e duração (s) por estrada entre todos os pares de pontos, modo DRIVE.
Google Routes API — computeRoutes	Geometria (polyline) da rota de cada veículo, para o mapa e para localizar as paradas de abastecimento ao longo dos trechos longos.
IBGE — API de localidades	Lista oficial de municípios por estado, usada para sugerir e validar as cidades do cadastro manual.
Catálogo interno (data/)	Cenário padrão, pool de 52 cidades de SP para o sorteio, grupos de medicamentos/insumos, regras de transporte e tipos de veículo.
Haversine x 1,3 (fallback)	Distância em linha reta multiplicada por um fator rodoviário de 1,3 e velocidade média de 60 km/h, usada quando não há chave do Google.

A Distance Matrix API e a Directions API, tradicionalmente usadas para esse fim, passaram ao status de legado no Google Maps Platform e não podem ser habilitadas em projetos novos. Por isso, a integração foi feita com a Routes API. A matriz é requisitada em blocos de 25 origens por 25 destinos, respeitando o limite de 625 elementos por chamada, e as geocodificações ficam em cache para reduzir custo e latência.

O cadastro de grupos de medicamentos/insumos e de veículos é fixo e foi definido a partir do enunciado:

Grupo	Refrigeração	Regra de transporte
A	Não	Não pode ser transportado junto com o grupo C (risco de contaminação).
B	Sim	Deve ser transportado sozinho, somente em caminhão com refrigeração.
C	Não	Não pode ser transportado junto com o grupo A (risco de contaminação).
D e E	Não	Sem restrição.

Veículo	Capacidade (kg)	Autonomia (km)	Refrigerado	Abastece na rota
Moto	15	140	Não	Sim
Carro	250	500	Não	Sim
Carro elétrico	300	300	Não	Não (autonomia sempre rígida)
Caminhão com refrigeração	1.000	600	Sim	Sim

Caminhão sem refrigeração	1.000	700	Não	Sim

Tipos de veículo. A frota padrão tem um veículo de cada tipo.

O sistema oferece três formas de montar o cenário:

- **Cenário padrão (5 cidades):** cidades fixas de SP, com São Paulo como hub (origem e destino), conforme a tabela abaixo;
- **Cidades aleatórias:** o usuário informa a quantidade (3 a 52) e o backend sorteia cidades do pool, atribuindo prioridade crítica com probabilidade de 30%, demanda entre 5 e 50 kg, um grupo de A a E e uma janela de horário. Todos os campos podem ser ajustados antes de otimizar;
- **Cadastro manual:** o usuário cadastra cidade a cidade (estado, município, grupo, prioridade, demanda, janela e papel: origem, destino, origem e destino ou entrega). O município é conferido na lista do IBGE antes da geocodificação, o que evita confundir uma cidade com um distrito homônimo de outro estado.

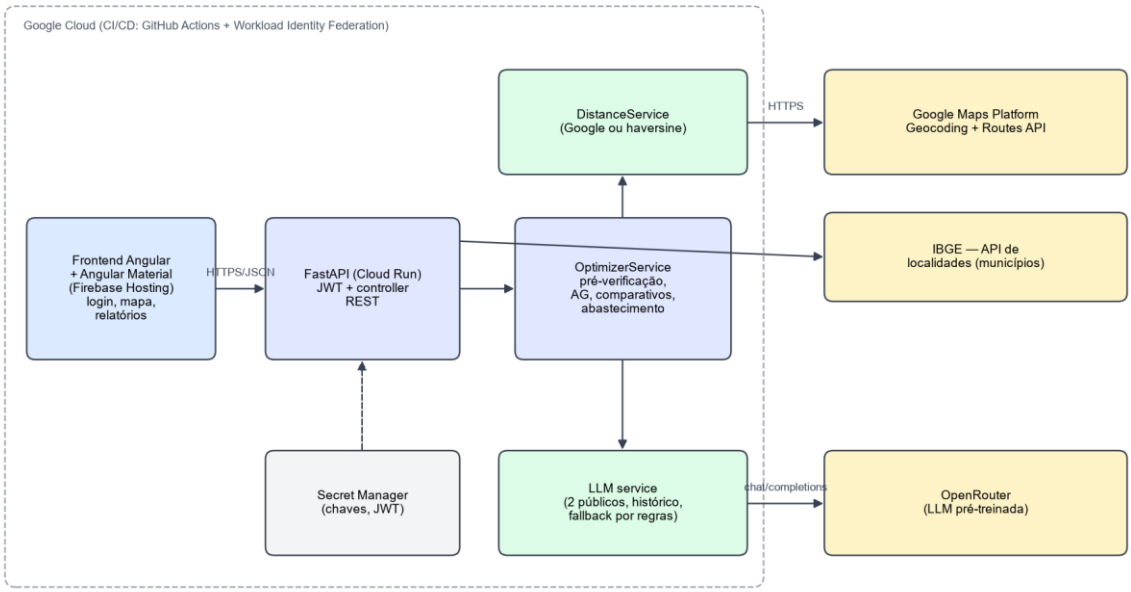
#	Cidade	Papel	Grupo	Prioridade	Demanda (kg)	Janela
0	São Paulo, SP	Hub (origem/destino)	—	—	0	—
1	Campinas, SP	Entrega	A	Crítica	30	08:00-12:00
2	Santos, SP	Entrega	C	Regular	40	09:00-17:00
3	Sorocaba, SP	Entrega	B	Crítica	25	07:00-11:00
4	São José dos Campos, SP	Entrega	D	Regular	35	13:00-18:00

Cidades do cenário padrão.

4. Arquitetura da solução

4.1 Visão geral

A solução segue uma arquitetura cliente-servidor publicada no Google Cloud. O frontend em Angular, hospedado no Firebase Hosting, consome uma API REST em Python (FastAPI) executada no Cloud Run, que concentra toda a lógica: autenticação, obtenção de distâncias, pré-verificação das restrições, execução do algoritmo genético, cálculo dos horários e abastecimentos, comparativos, geometrias das rotas e geração dos textos pela LLM.



Arquitetura da solução: frontend, API, serviços internos e serviços externos.

4.2 Fluxo de uma otimização

- O usuário faz login e recebe um token JWT, enviado em todas as chamadas seguintes;
- O frontend envia as cidades (com grupo, prioridade, demanda e janela), origem, destino, frota, horários, restrições, parâmetros do AG e os identificadores das execuções anteriores da sessão;
- O DistanceService monta a matriz NxN de distâncias e tempos (Routes API ou fallback haversine);
- A pré-verificação procura restrições rígidas impossíveis (ex.: carga refrigerada sem caminhão refrigerado); se houver, a API responde 422 com os motivos, sem executar o AG;
- O OptimizerService executa o algoritmo genético e compara a melhor solução com o vizinho mais próximo e com uma solução aleatória média;
- Para cada veículo usado, calcula horários, esperas, atrasos e paradas para abastecimento, e obtém a geometria real da rota (computeRoutes); paradas no meio de trechos longos são localizadas na geometria e nomeadas por reverse geocoding;
- O resultado, sem as geometrias, é enviado em JSON para a LLM em duas chamadas paralelas: análise técnica (com o histórico das execuções anteriores com as mesmas cidades) e instruções para os motoristas;

- O resultado completo fica em memória com um identificador, usado nas perguntas em linguagem natural e no histórico.

4.3 Backend e endpoints

O backend é um projeto Python 3.13 com dependências em requirements.txt e imagem Docker. As chaves ficam em variáveis de ambiente (arquivo .env no desenvolvimento e Secret Manager na nuvem), nunca no código. A documentação interativa (Swagger/OpenAPI) é gerada pelo FastAPI em /docs. Todas as rotas /api, exceto o login, exigem token.

Método	Rota	Descrição
POST	/api/auth/login	Autentica o usuário e devolve o token de acesso (JWT, validade de 8 horas)
GET	/api/auth/me	Usuário do token
GET	/api/health	Status e provedores ativos (Google/haversine, LLM habilitada)
GET	/api/catalog	Grupos de medicamentos/insumos, regras de transporte e tipos de veículo
GET	/api/cities/default	Cidades do cenário padrão
GET	/api/cities/random?count=N	Sorteia N cidades com prioridade, demanda, grupo e janela
GET	/api/geo/states	Estados brasileiros (cadastro manual)
GET	/api/geo/states/{uf}/cities	Municípios do estado (IBGE)
POST	/api/cities/geocode	Coordenadas de um município validado no IBGE
POST	/api/routes/optimize/default	Otimiza o cenário padrão
POST	/api/routes/optimize	Otimiza as cidades informadas, com origem e destino escolhidos
POST	/api/routes/ask	Pergunta em linguagem natural sobre um resultado (público técnico ou motorista)

Exemplo de requisição de otimização (trecho):

```
POST /api/routes/optimize
{
  "locations": [{"name": "Campinas, SP", "lat": -22.90, "lng": -47.06,
                 "priority": "critical", "demand": 30,
                 "product_group": "A",
                 "window_start": "08:00", "window_end": "12:00"}, ...],
  "origin_index": 0, "destination_index": 0,
  "vehicles": [{"type": "moto"}, {"type": "carro"},
               {"type": "caminhao_refrigerado"}],
  "schedule": {"departure_time": "07:00", "service_minutes": 15,
               "refuel_enabled": true, "refuel_reserve_pct": 15,
               "refuel_minutes": 15},
  "ga": {"population_size": 100, "generations": 300,
         "mutation_probability": 0.3, "tournament_size": 3,
         "elite_size": 2, "priority_weight": 0.5, "seed": 42,
         "stagnation_generations": 100},
  "constraints": {"capacity": "hard", "autonomy": "hard",
                  "priority": "soft", "time_window": "soft",
```

```
                    "soft_penalty": 50},
  "use_llm": true,
  "previous_result_ids": ["..."]
}
```

A resposta traz, por veículo, a sequência de paradas com horários de chegada, espera e atraso, as paradas para abastecimento (na cidade ou no caminho, com local e horário), carga, distância, grupos transportados e indicadores de cada restrição, além dos totais, das violações soft aceitas, do histórico do melhor fitness por geração, do comparativo com as outras abordagens e dos dois textos da LLM.

4.4 Frontend

O frontend é uma aplicação Angular 20 com componentes standalone e Angular Material, em tema escuro com a identidade visual do grupo e layout responsivo (as tabelas viram cartões em telas estreitas). Após o login, a tela de otimização oferece:

- Os três modos de cenário (padrão, aleatório e cadastro manual), com as regras de transporte em destaque;
- Parâmetros separados em grupos (veículos e operação, restrições e algoritmo genético), cada um com tooltip explicando o efeito;
- Grade da frota com tipo por veículo, geração aleatória e botão de exclusão;
- Indicação das etapas da otimização enquanto a API processa;
- Resultado com indicadores, mapa interativo (Leaflet + OpenStreetMap) com rotas coloridas por veículo, paradas numeradas e marcadores ⛽ de abastecimento, rotas por veículo com horários e abastecimentos, comparativo, gráfico de convergência e as duas abas da LLM (análise técnica e instruções para motoristas), cada uma com seu chat de perguntas.

4.5 Segurança

A API exige autenticação em todas as rotas de negócio. O usuário é configurado por variáveis de ambiente (AUTH_USERNAME e AUTH_PASSWORD) e o login devolve um JWT assinado com HS256, com validade configurável (padrão de 8

horas). O token é guardado no navegador e enviado por um interceptor HTTP; ao expirar, o usuário volta para a tela de login. O CORS aceita apenas as origens do frontend publicado, e nenhuma chave é versionada: o .gitignore exclui o .env e, na nuvem, as chaves ficam no Secret Manager.

4.6 Implantação na nuvem e CI/CD

A aplicação é publicada no Google Cloud com dois workflows do GitHub Actions:

- **CI:** a cada push e pull request, roda os testes do backend (pytest) e o build de produção do frontend;
- **Deploy:** constrói a imagem Docker do backend, envia ao Artifact Registry, publica no Cloud Run (região southamerica-east1, 1 vCPU, 512 MiB, escala de 0 a 1 instância), confere o /api/health e publica o frontend no Firebase Hosting com a URL da API.

O GitHub se autentica no Google sem chave de conta de serviço, por Workload Identity Federation: o workflow recebe um token OIDC do GitHub, que o Google troca por uma credencial temporária, aceita somente para este repositório e para as branches autorizadas. Como o Cloud Run escala até zero e o Firebase Hosting tem cota gratuita, o custo de um projeto acadêmico fica próximo de zero; um alerta de orçamento avisa caso os gastos passem de R$ 50. Aplicação publicada: https://rotas-medicas-fiap-70524.web.app.

5. Implementação do algoritmo genético

5.1 Código base e modificações

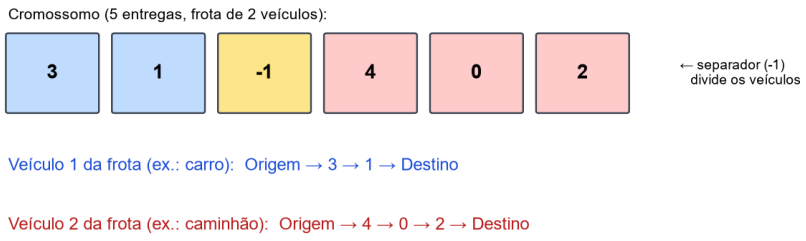
O ponto de partida foi o repositório FIAP/genetic_algorithm_tsp, que resolve o TSP com coordenadas cartesianas, distância euclidiana, crossover OX, mutação por troca de vizinhos e visualização em Pygame. A tabela resume as mudanças feitas para o contexto hospitalar.

Aspecto	Código base (TSP)	Este projeto (VRP médico)
Indivíduo	Lista de coordenadas (x, y)	Permutação de entregas + separadores; a rota i é do veículo i da frota
Distância	Euclidiana em pixels	Distância e tempo reais por estrada (Google Routes API)
Fitness	Distância total do ciclo	Distância + prioridade + penalidades soft e hard
População inicial	Aleatória (TODO: heurística)	Aleatória reparada + 1 indivíduo do vizinho mais próximo

Seleção	Top 10 ou roleta por 1/fitness	Torneio com k configurável
Crossover	Order Crossover (OX)	OX mantido, compatível com separadores
Mutação	Troca de genes adjacentes (TODO: inversão)	Troca de dois genes ou inversão de segmento (2-opt)
Reparo	—	Move entregas que violam contaminação/refrigeração para o veículo compatível mais barato
Elitismo	1 indivíduo	Configurável (padrão: 2)
Parada	Número fixo de gerações	Limite de gerações ou estagnação (padrão: 100 gerações sem melhora)
Reprodutibilidade	—	Semente configurável (padrão: 42)
Rota	Ciclo fechado	Origem e destino definidos pelo usuário (aberta ou ida e volta)
Visualização	Pygame	Mapa web (Leaflet) com rotas reais e gráfico de convergência

5.2 Representação genética

Cada indivíduo é uma permutação que contém os índices (não negativos) dos pontos de entrega e K-1 separadores com valores negativos únicos (-1, -2, ...), em que K é o número de veículos da frota. Os separadores dividem a permutação em K rotas, e a i-ésima rota é feita pelo i-ésimo veículo cadastrado, o que permite frotas heterogêneas: mover uma entrega para outro trecho do cromossomo é trocá-la de veículo. A origem e o destino não fazem parte do cromossomo, pois são fixos e adicionados na decodificação. Como todos os genes são únicos, os operadores de permutação nunca geram indivíduos com entregas repetidas ou faltantes.



Exemplo de cromossomo com 5 entregas e frota de 2 veículos: o separador -1 divide as entregas.

```
def split_routes(chromosome):
    routes = [[]]
    for gene in chromosome:
        if gene < 0:            # separador: inicia a rota do próximo veículo
            routes.append([])
        else:
            routes[-1].append(gene)
    return routes
```

5.3 População inicial

A população inicial é formada por permutações aleatórias, já passadas pelo operador de reparo, e por um indivíduo construído pela heurística do vizinho mais próximo, que resolve um dos TODOs do código base. A heurística preenche os veículos do maior para o menor, respeita capacidade, autonomia, refrigeração e compatibilidade de grupos, e reserva o caminhão refrigerado para a carga que precisa de refrigeração. Com o elitismo, o AG nunca termina pior que essa semente.

5.4 Seleção

A seleção é feita por torneio: k indivíduos (padrão k = 3) são sorteados e o de menor fitness é escolhido como pai. Em relação à roleta do código base, o torneio não depende da escala do fitness, o que é essencial porque as penalidades hard elevam o fitness de soluções inviáveis em várias ordens de grandeza. Os melhores indivíduos (elitismo, padrão 2) são copiados diretamente para a geração seguinte.

5.5 Crossover

O Order Crossover (OX) do código base foi mantido: um segmento contínuo do primeiro pai é copiado para o filho e as posições restantes são preenchidas com os genes do segundo pai, na ordem em que aparecem. Como os separadores são genes como os demais, o OX também herda a forma como os pais dividem as entregas entre os veículos.

5.6 Mutação

Com probabilidade igual à taxa de mutação, o filho sofre um de dois operadores, sorteados com igual chance:

- Troca (swap): dois genes quaisquer trocam de posição, o que pode mover uma entrega de um veículo para outro;
- Inversão de segmento: um trecho do cromossomo é invertido, equivalente a um movimento 2-opt, eficaz para desfazer cruzamentos na rota (TODO do código base).

5.7 Operador de reparo

As regras sanitárias são as restrições mais difíceis para a busca: uma única entrega do grupo B num veículo comum, ou um par A e C no mesmo veículo, torna a solução inviável. Depois do crossover e da mutação, cada filho passa por um reparo: em cada veículo, as entregas são lidas na ordem, e a que exige refrigeração num veículo comum, ou que é incompatível com algo já carregado, é retirada e reinserida no veículo compatível e na posição que menos aumentam a distância, preferindo onde ainda cabe na capacidade e na autonomia. Se não houver veículo compatível, a entrega volta ao veículo original e a violação continua penalizada.

5.8 Função fitness

O fitness é minimizado e combina quatro componentes:

```
fitness = distância_total
        + w_prioridade × Σ (km acumulado até cada entrega crítica)
        + penalidade_soft × Σ (violações das restrições soft)
        + 1.000.000 × Σ (violações das restrições hard)
```

A distância total soma, para cada veículo usado, o trajeto origem → entregas → destino. O termo de prioridade penaliza entregas críticas que acontecem tarde na rota. As violações são medidas em unidades naturais: kg acima da capacidade, km acima da autonomia, minutos de atraso nas janelas, entregas críticas depois de regulares e entregas em conflito de contaminação ou refrigeração. Cada restrição entra no termo soft (com a penalidade configurada, padrão 50 km equivalentes por unidade) ou no termo hard (fator 1.000.000), conforme o modo escolhido pelo usuário. O fator hard torna qualquer violação rígida mais cara que qualquer ganho de distância possível, de modo que soluções viáveis sempre dominam as inviáveis.

Na avaliação de cada rota, o sistema simula a viagem: soma os tempos de deslocamento, espera a janela abrir quando chega cedo, registra o atraso quando chega tarde, adiciona o tempo de descarga (padrão 15 min) e os abastecimentos (seção 6.4). Assim, os horários exibidos na tela e enviados à LLM são os mesmos usados no fitness.

5.9 Critério de parada e reprodutibilidade

Além do número máximo de gerações, o AG para quando o melhor fitness passa um número configurável de gerações sem melhorar (padrão 100; 0 desliga o critério). A resposta informa quantas gerações foram executadas e se a parada foi por estagnação. Todas as decisões aleatórias usam um gerador com semente configurável (padrão 42): com a mesma semente e os mesmos dados, o resultado é idêntico, o que permite comparar parâmetros de forma justa. Sem semente, cada execução explora caminhos diferentes.

5.10 Parâmetros

Parâmetro	Padrão	Faixa aceita
Tamanho da população	100	10 a 1000
Número de gerações	300	1 a 5000
Probabilidade de mutação	0,3	0 a 1
Tamanho do torneio (k)	3	2 a 20
Elitismo	2	1 a 50
Peso da prioridade	0,5	≥ 0
Semente	42	≥ 0 ou vazia (aleatória)
Parada por estagnação	100 gerações	0 (desligada) ou mais
Penalidade soft	50	≥ 0
Horário de saída / descarga	07:00 / 15 min	HH:MM / 0 a 240 min
Abastecimento: reserva / duração	15% / 15 min	0 a 50% / 0 a 120 min
Frota	1 veículo de cada tipo	1 a 10 veículos

6. Estratégias para lidar com as restrições

6.1 Restrições flexíveis (soft) e rígidas (hard)

Capacidade, autonomia, prioridade e janelas de horário podem ser configuradas como soft ou hard. No modo hard, a violação recebe o fator de 1.000.000 e a solução é considerada inviável; se a pré-verificação detectar que a restrição é impossível de cumprir, a API responde 422 com os motivos (por exemplo, a cidade e a distância que excedem a autonomia de todos os veículos aptos). No modo soft, a violação é aceita com custo proporcional à penalidade, e a resposta lista as violações aceitas. Contaminação, refrigeração e a autonomia do carro elétrico são sempre rígidas, por segurança sanitária e por limite físico.

6.2 Prioridades

Por padrão, a prioridade é um objetivo secundário: o termo de prioridade soma, para cada entrega crítica, a distância acumulada do veículo até ela, multiplicada por um peso configurável. Com peso zero, o sistema minimiza só a distância; com pesos maiores, antecipa as entregas críticas. No modo hard, cada entrega crítica feita depois de uma regular no mesmo veículo é uma violação rígida. O efeito do peso é medido na seção 8.5.

6.3 Capacidade de carga

A carga de cada veículo é a soma das demandas das entregas atribuídas a ele, comparada à capacidade do tipo de veículo. Antes de executar o AG, a pré-verificação confere se cada entrega cabe no maior veículo apto, se a demanda total cabe na frota e se a carga refrigerada cabe nos caminhões refrigerados.

6.4 Autonomia e abastecimento

Todos os veículos saem de tanque cheio. Com o abastecimento habilitado, antes de cada trecho o sistema verifica se o percurso deixaria a autonomia abaixo da reserva (padrão 15%) e, nesse caso, o veículo abastece na cidade em que está. Se o próprio trecho for maior que a autonomia útil, o veículo abastece no caminho quantas vezes forem necessárias, sempre ao atingir a reserva, e nunca chega com o tanque vazio. Cada abastecimento soma o tempo configurado (padrão 15 min) aos horários seguintes. O ponto de cada parada no caminho é localizado sobre a geometria real da rota e nomeado pelo município (reverse geocoding); sem Google, aparece como "km X do trecho A → B".

O carro elétrico não recarrega na rota: sua autonomia vale para o percurso inteiro, contando a volta ou o trecho até o destino, e é sempre rígida. Com o abastecimento desligado, a mesma regra vale para todos os veículos.

6.5 Janelas de horário

Cada entrega tem uma janela de recebimento. A simulação parte do horário de saída e, em cada parada, soma o deslocamento e os abastecimentos: se o veículo chega antes do início, espera; se chega depois do fim, registra o atraso em minutos, que é a violação da restrição. No modo hard, a pré-verificação

rejeita entregas que não podem ser atendidas no prazo nem indo direto da origem.

6.6 Grupos de medicamentos: contaminação e refrigeração

A violação de contaminação conta, para cada par incompatível presente no mesmo veículo, o menor dos dois grupos, o que dá ao AG um sinal gradual de quantas entregas estão no veículo errado. A de refrigeração conta as entregas do grupo B em veículos sem refrigeração. As duas são sempre rígidas e, além da penalidade, são tratadas pelo operador de reparo (seção 5.7) e pela heurística do vizinho mais próximo. Quando a carga refrigerada não cabe nos caminhões refrigerados, a API explica quais cidades ficaram de fora.

6.7 Frota heterogênea e múltiplos veículos

A divisão entre veículos é codificada nos separadores do cromossomo e otimizada junto com a ordem das entregas. Como a posição da rota no cromossomo define o tipo de veículo, o AG decide também qual veículo atende cada região: motos para entregas leves e próximas, caminhões para cargas grandes. Um veículo sem entregas não sai do hub e não contribui para a distância, e a LLM é informada de quais veículos ficaram ociosos.

6.8 Origem e destino distintos

O usuário pode escolher origem e destino diferentes, como um centro de distribuição de saída e um hospital de chegada. Os dois pontos são removidos do conjunto de entregas e adicionados no início e no fim da rota de cada veículo. Quando coincidem, a rota é de ida e volta, como no código base.

7. Integração com LLMs

7.1 Abordagem

A integração usa o OpenRouter, um gateway compatível com a API de Chat Completions da OpenAI que dá acesso a diversas LLMs pré-treinadas com a mesma interface. O modelo é configurável (OPENROUTER_MODEL; padrão anthropic/claude-haiku-5.5), com modelos de reserva opcionais que o OpenRouter tenta em ordem quando o principal falha. O cliente repete a chamada em caso de limite de uso (429) ou indisponibilidade (502/503),

respeitando o Retry-After; se a resposta vier vazia ou cortada pelo limite de tokens, tenta de novo com o dobro do limite. O esforço de raciocínio é configurado como baixo, para que modelos com raciocínio não gastem todo o orçamento antes de responder.

7.2 Dois públicos: análise técnica e motoristas

Cada otimização gera dois textos, em chamadas paralelas, com prompts de sistema diferentes:

Texto	Público	Seções exigidas
Análise técnica	Equipe que testa e calibra o sistema	Diagnóstico da execução; comparação com execuções anteriores (quando houver); qualidade da solução (tabela comparativa); convergência; restrições, frota e operação; sugestões de ajuste de parâmetros (tabela com valor atual, sugerido, por quê e efeito esperado); próximos testes recomendados
Instruções para motoristas	Motoristas e equipes de entrega	Resumo do dia; instruções por veículo (o que carregar, paradas com horário e janela, abastecimentos, cuidados); veículos que não saem; cuidados gerais (separação de grupos, cadeia de frio, conferência, atrasos)

Os dois textos atendem aos usos pedidos no enunciado: instruções detalhadas para motoristas e equipes, relatório sobre a eficiência das rotas e sugestões de melhoria com base nos padrões identificados.

7.3 Prompts

Foram aplicadas técnicas de prompt engineering para obter respostas fiéis e úteis:

- Papel definido no prompt de sistema: especialista em VRP e algoritmos genéticos (técnico) ou coordenador de logística hospitalar (motoristas);
- Ancoragem nos dados: usar exclusivamente o JSON fornecido, sem inventar distâncias, horários, cidades ou veículos;
- Estrutura de saída fixa, com seções obrigatórias em Markdown;
- Sugestões com evidência: toda sugestão técnica cita o valor atual do parâmetro, o sugerido e o porquê, apoiado em convergência, comparativo, violações ou ocupação da frota; se os parâmetros estiverem adequados, o modelo deve dizer isso;

- Regras de domínio: destacar entregas críticas, separação dos grupos, cadeia de frio, janelas e abastecimentos;
- Linguagem adequada ao público: termos técnicos só na análise técnica;
- Temperatura baixa (0,3) para reduzir variação e invenção.

Trecho do prompt de sistema da análise técnica:

```
Você é um especialista em otimização de rotas (VRP) e algoritmos genéticos,
analisando uma execução do sistema para a equipe de desenvolvimento.
Regras:
- Use EXCLUSIVAMENTE os dados do JSON fornecido. Não invente distâncias,
  horários, cidades, cargas ou veículos.
- Toda sugestão deve citar o valor atual do parâmetro, o valor sugerido e
  o porquê, apoiado em evidências dos dados.
- Se os parâmetros atuais já parecem adequados, diga isso; não sugira
  mudanças sem evidência.
```

7.4 Contexto enviado à LLM

O backend envia um JSON compacto, sem as geometrias das rotas, com: os parâmetros executados (AG, restrições e horários); a frota inteira, inclusive veículos não usados, com ocupação de capacidade e autonomia; as cidades com papel, grupo, prioridade, demanda e janela; as rotas com horários, esperas, atrasos e paradas para abastecimento (local, trecho e horário); um resumo da convergência (geração da última melhora, gerações estagnadas, se parou por estagnação); o comparativo com as outras abordagens; as regras de transporte e as violações soft aceitas.

7.5 Histórico de execuções na análise técnica

Calibrar o AG é um processo iterativo: o usuário executa, lê as sugestões, ajusta parâmetros e executa de novo. Para que a análise técnica compare as execuções e não sugira um valor já testado, o frontend envia os identificadores das execuções anteriores da sessão, e o backend usa as que têm as mesmas cidades, origem e destino (até as 5 mais recentes). Para cada uma, a LLM recebe os parâmetros, os resultados (distância, fitness, viabilidade, atrasos, abastecimentos, convergência e tempo), a lista do que mudou em relação à anterior (por exemplo, "mutation_probability: 0.3 → 0.5") e o trecho de sugestões que ela mesma fez. O relatório ganha a seção "Comparação com execuções anteriores", com uma tabela por execução e a análise do efeito de cada

mudança. Quando o usuário muda alguma cidade, a origem ou o destino, o histórico recomeça. As instruções para motoristas não recebem o histórico.

7.6 Perguntas em linguagem natural

Cada aba da LLM tem seu próprio chat. O endpoint /api/routes/ask recebe o identificador da otimização, a pergunta e o público. Perguntas técnicas ("Vale aumentar a mutação neste cenário?") recebem o contexto completo e o histórico de execuções; perguntas de motoristas ("Que horas chego em Campinas e o que entrego lá?") são respondidas em linguagem simples, e a LLM deve dizer quando a informação não está nos dados.

7.7 Avaliação da qualidade e fallback

A qualidade das respostas foi avaliada com os seguintes critérios:

- Fidelidade: os números citados (km, horários, kg) coincidem com o JSON da otimização;
- Completude: todas as seções exigidas aparecem e todos os veículos usados têm instruções;
- Coerência das sugestões técnicas com as evidências (convergência, violações, frota ociosa) e com o histórico;
- Destaque das entregas críticas, das regras de transporte e dos abastecimentos;
- Clareza para o público-alvo.

Exemplo real. O cenário padrão foi otimizado duas vezes com distâncias reais do Google e o modelo anthropic/claude-haiku-5.5: a primeira com os parâmetros padrão e a segunda com população 150 e mutação 0,5, enviando a primeira como execução anterior. As duas chegaram à mesma solução: 692,2 km, três veículos (carro para Campinas e São José dos Campos, carro elétrico para Santos e caminhão refrigerado para Sorocaba), nenhuma violação e parada por estagnação na geração 100. Trechos da análise técnica da segunda execução:

```
## Comparação com execuções anteriores
| Execução  | Parâmetros alterados        | Distância | Fitness | AG    |
| 1         | Linha base (pop. 100, 0,3)  | 692,2     | 787,53  | 0,5 s |
| 2 (atual) | Pop. 150; mutação 0,5       | 692,2     | 787,53  | 1,6 s |
- Nenhuma das mudanças melhorou o resultado. Aumentar a população e
  a mutação manteve a mesma solução e triplicou o tempo de execução.
```

```
## Restrições, frota e operação
- As regras de transporte forçam pelo menos 3 veículos: B exige
  caminhão refrigerado sozinho, e A e C não podem compartilhar
  veículo. O uso de 3 veículos é estrutural.
- Carro: espera de 148,1 min em São José dos Campos (chega às 10:32,
  janela abre às 13:00). É a maior ineficiência da solução.

## Sugestões de ajuste de parâmetros
| Parâmetro       | Atual | Sugerido | Por quê                       |
| seed            | 42    | 1 a 10   | Geração 0 não distingue ótimo |
|                 |       |          | de população enviesada        |
| departure_time  | 07:00 | 09:00    | Espera de 148 min; sugerida   |
|                 |       |          | na execução 1, não aplicada   |
| population_size | 150   | 50       | 100 e 150: mesmo resultado    |
```

Trecho das instruções para o motorista do carro, na mesma execução:

```
### Veículo 2 — Carro
1. Campinas, SP ⚠️ CRÍTICA — chegada 08:19, janela 08:00-12:00,
   entregar 30 kg do grupo A. Não há espera. Saia por volta das 08:34.
2. São José dos Campos, SP — chegada 10:32, janela 13:00-18:00.
   ⏳ Você vai esperar cerca de 2 horas e 30 minutos (148 min).
⛽ Abastecimento: não é necessário (334 km de 500 km).
- Este carro leva só os grupos A e D. Não carregue o grupo C.
```

Critério	Avaliação do exemplo
Fidelidade	Distâncias, fitness, cargas, horários e esperas coincidem com o JSON. A análise apontou como "inconsistência" a melhoria sobre o vizinho mais próximo ser 3,43% na execução 1 e 2,34% na 2; os dois valores estão certos, porque a matriz do Google variou entre as chamadas (o vizinho mais próximo deu 720,1 e 711,1 km). A LLM percebeu a diferença, mas errou a causa
Completude	Todas as seções exigidas, incluindo a comparação com a execução anterior; todos os veículos usados têm instruções e os ociosos foram listados
Coerência das sugestões	Não repetiu população nem mutação já testadas; propôs variar a semente e lembrou que a mudança de horário sugerida na execução 1 não foi aplicada; manteve a parada por estagnação com justificativa
Regras e abastecimento	Explicou por que são necessários três veículos (grupos A/C e B) e verificou que não há abastecimento; as instruções reforçam a separação dos grupos e a cadeia de frio
Clareza para o público	Análise técnica com termos de AG e tabelas; instruções em linguagem simples, com horários e o que fazer em cada parada

Avaliação do exemplo segundo os critérios definidos.

O exemplo mostra o valor do histórico: a LLM concluiu que população e mutação não eram o gargalo e mudou de direção, em vez de sugerir de novo os mesmos parâmetros. Também mostra por que as respostas devem ser revisadas: a leitura da diferença entre as execuções estava errada, embora os números estivessem corretos.

Quando a chave do OpenRouter não está configurada ou todas as tentativas falham, o backend gera os dois textos por regras: a análise técnica traz diagnóstico, tabela comparativa, convergência, comparação com as execuções anteriores e sugestões heurísticas (por exemplo, mais gerações quando a última melhora foi no fim, ou retirar veículos ociosos), sem repetir valores já testados; as instruções para motoristas listam cargas, paradas, horários e abastecimentos. A resposta indica se o texto veio da LLM ou das regras.

8. Experimentos e resultados

8.1 Configuração dos experimentos

Os experimentos foram refeitos com o modelo da release atual: frota padrão com um veículo de cada tipo, grupos de medicamentos/insumos, janelas de horário, abastecimento habilitado (reserva de 15%, 15 min por parada), saída às 07:00, descarga de 15 min e restrições nos padrões da API (capacidade e autonomia hard, prioridade e janelas soft, penalidade soft 50). Foram avaliados três cenários e quatro configurações do AG, com 5 execuções independentes (sementes diferentes) por combinação. Para que os resultados fossem reprodutíveis sem consumo de API, as distâncias vieram do provedor de fallback (haversine x 1,3, 60 km/h); na aplicação, com a chave configurada, as mesmas rotinas usam as distâncias e os tempos reais do Google Maps.

Cenário	Nº de pontos	Hub	Demanda total	Críticas	Entregas por grupo
Padrão	5	São Paulo	130 kg	2	A:1 / B:1 / C:1 / D:1
15 cidades	15	Hortolândia	423 kg	4	A:4 / B:1 / C:4 / D:3 / E:2
25 cidades	25	Guarujá	707 kg	4	A:3 / B:6 / C:5 / D:5 / E:5

Cenários avaliados (frota padrão de 5 veículos em todos).

Config.	População	Gerações	Prob. mutação
A	50	200	0,1
B	100	300	0,3
C	200	500	0,5
D	100	300	0,8

Configurações do AG avaliadas (torneio k = 3, elitismo = 2, peso de prioridade = 0,5, sem parada por estagnação, para comparar o número de gerações).

8.2 Resultados por configuração

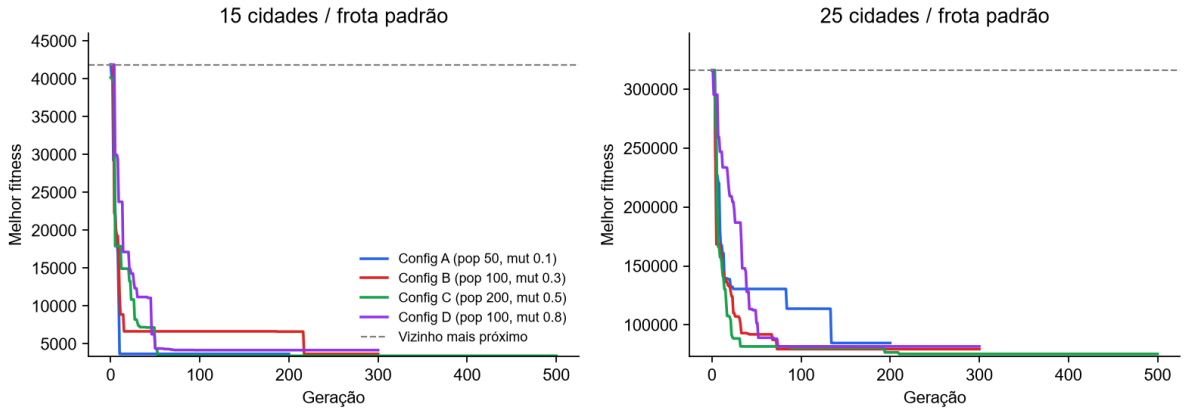
Config.	Fitness médio	Desvio	Melhor fitness	Distância média (km)	Atraso médio (min)	Viáveis	Tempo médio (s)
A	7.209,9	4.001,3	3.669,9	2.311,4	90	5/5	2,1
B	4.537,2	1.098,0	3.669,9	2.383,2	35	5/5	6,5
C	3.718,4	244,5	3.416,1	2.451,3	20	5/5	26,7
D	4.239,4	89,8	4.166,1	2.339,1	33	5/5	8,7

Resultados no cenário de 15 cidades (5 execuções por configuração).

Config.	Fitness médio	Desvio	Melhor fitness	Distância média (km)	Atraso médio (min)	Viáveis	Tempo médio (s)
A	105.677,7	14.879,4	84.573,7	3.773,7	2.027	5/5	3,5
B	83.418,7	3.400,0	79.386,4	3.857,1	1.582	5/5	9,8
C	80.371,9	2.535,5	75.300,8	3.853,7	1.521	5/5	26,4
D	84.158,9	5.037,9	81.639,7	3.810,0	1.597	5/5	6,3

Resultados no cenário de 25 cidades (5 execuções por configuração).

Em ambos os cenários, a configuração C (população 200, 500 gerações, mutação 0,5) obteve o menor fitness médio e o melhor resultado individual, com dispersão pequena entre as sementes: no cenário de 15 cidades, desvio de 244,5, contra 4.001,3 da configuração A. A configuração A, com população pequena e pouca mutação, converge cedo e depende muito da semente: em algumas execuções encontra boas soluções, em outras fica presa com atrasos maiores. A configuração D, com mutação alta (0,8), teve a menor dispersão no cenário de 15 cidades, mas em torno de um resultado pior (fitness médio de 4.239,4), com um quarto a um terço do tempo de C. Todas as execuções terminaram viáveis, ou seja, sem violar contaminação, refrigeração, capacidade e autonomia; o que diferencia as configurações é quanto conseguem reduzir os atrasos, tratados como restrição soft. O custo computacional cresce com população × gerações: no cenário de 15 cidades, C leva em média 26,7 s, contra 6,5 s de B.



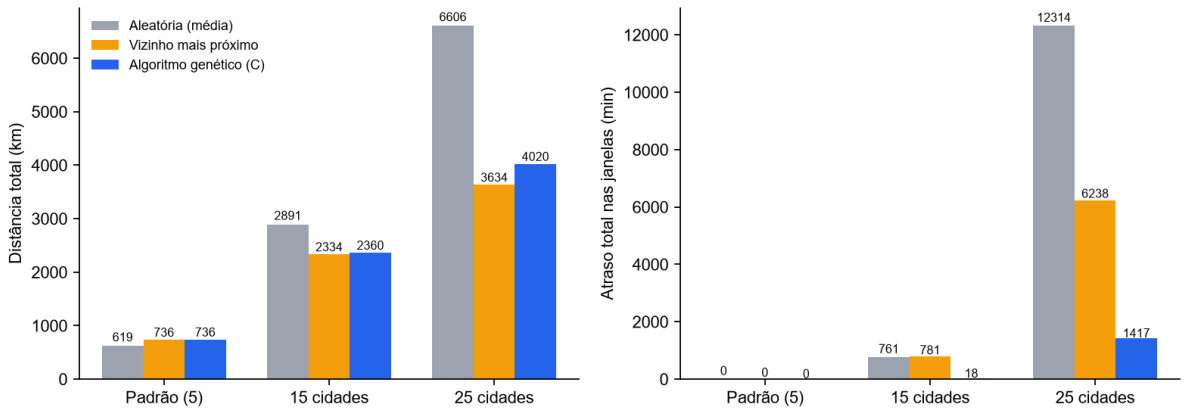
Evolução do melhor fitness por geração (melhor execução de cada configuração). A linha tracejada é o fitness do vizinho mais próximo.

O gráfico mostra que todas as configurações partem do fitness do vizinho mais próximo, graças à semente heurística, e o reduzem rapidamente nas primeiras dezenas de gerações, quando os atrasos nas janelas são eliminados. Depois disso aparecem patamares longos interrompidos por melhorias pontuais, típicos de problemas com restrições de horário: uma pequena troca de ordem pode liberar uma janela inteira.

8.3 Comparativo com outras abordagens

Cenário	Abordagem	Distância (km)	Atraso (min)	Fitness	Viável
Padrão	Aleatória (média)	619,0	0	42.381.553,2	Não
Padrão	Vizinho mais próximo	736,3	0	845,3	Sim
Padrão	Algoritmo genético (C)	736,3	0	845,3	Sim
15 cidades	Aleatória (média)	2.891,4	761	418.128.772,0	Não
15 cidades	Vizinho mais próximo	2.334,3	781	41.861,8	Sim
15 cidades	Algoritmo genético (C)	2.360,1	18	3.416,1	Sim
25 cidades	Aleatória (média)	6.605,6	12.314	1.139.858.107,8	Não
25 cidades	Vizinho mais próximo	3.634,5	6.238	316.106,0	Sim
25 cidades	Algoritmo genético (C)	4.020,4	1.417	75.300,8	Sim

Comparativo entre o algoritmo genético (melhor execução da configuração C), o vizinho mais próximo e uma solução aleatória de fitness médio.



Distância total e atraso total nas janelas de horário por abordagem em cada cenário.

No cenário de 15 cidades, o AG reduziu o atraso total de 781 para 18 minutos com praticamente a mesma distância (2.360,1 km contra 2.334,3 km do vizinho mais próximo, 1,1% a mais), e o fitness médio caiu 91,1%. No de 25 cidades, os atrasos caíram 77,3% (6.238 para 1.417 minutos), com 9,6% a mais de distância. O vizinho mais próximo minimiza só a distância do próximo passo e ignora os horários; o AG troca alguns quilômetros por uma ordem de visitas compatível com as janelas. Os atrasos que restam no cenário de 25 cidades vêm das janelas sorteadas, muitas concentradas na manhã, que uma frota de cinco veículos não consegue atender a tempo. As soluções aleatórias violam regras sanitárias e de capacidade e não são viáveis. Esses resultados confirmam H1.

No cenário padrão, o AG e o vizinho mais próximo chegam à mesma solução (736,3 km, três veículos), que as quatro configurações reproduzem em todas as execuções. A solução aleatória mostrada é mais curta (619,0 km) porque coloca grupos incompatíveis no mesmo veículo e a carga refrigerada em veículo comum: as regras sanitárias têm um custo de distância, e o sistema nunca o evita violando-as.

8.4 Critério de parada por estagnação e reprodutibilidade

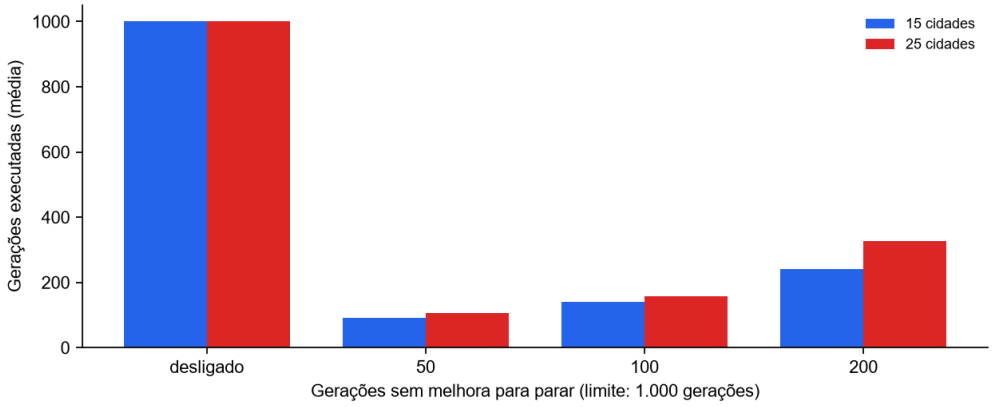
Para medir o critério de parada (hipótese H5), os cenários de 15 e 25 cidades foram executados com limite de 1.000 gerações (população 100, mutação 0,3) e diferentes números de gerações sem melhora para parar, 5 execuções cada.

Cenário	Parada por estagnação	Gerações executadas	Fitness médio	Melhor fitness	Tempo médio (s)
15 cidades	desligada	1.000	5.999,4	3.669,9	12,6
15 cidades	50 gerações	91	7.980,5	4.349,3	1,2

15 cidades	100 gerações	141	7.980,5	4.349,3	2,3
15 cidades	200 gerações	241	7.980,5	4.349,3	4,0
25 cidades	desligada	1.000	82.325,2	79.386,4	14,1
25 cidades	50 gerações	107	91.919,8	80.542,7	1,6
25 cidades	100 gerações	157	91.919,8	80.542,7	2,2
25 cidades	200 gerações	326	89.306,0	79.386,4	4,6

Efeito do critério de parada por estagnação (limite de 1.000 gerações).

Com o padrão de 100 gerações, a execução parou em média na geração 141 (15 cidades) e 157 (25 cidades), com redução de 81,6% e 84,3% no tempo. O custo foi um fitness médio maior: 24,8% no cenário de 15 cidades e 10,4% no de 25. Os resultados iguais com 50 e 100 gerações mostram patamares longos: o AG passa mais de 100 gerações sem melhorar e volta a melhorar mais tarde, o que só acontece quando ele continua rodando. H5 é confirmada em parte: o critério reduz muito o tempo, mas em cenários com muitas janelas de horário vale aumentar o valor ou desligá-lo. Por isso a resposta da API informa se a execução parou por estagnação, e a análise técnica da LLM usa essa informação nas sugestões.



Gerações executadas em média para cada valor do critério de parada.

Para a reprodutibilidade, o cenário de 15 cidades foi executado duas vezes com a semente 42: as duas execuções deram exatamente o mesmo fitness (3.669,9) e as mesmas rotas. Sem semente, duas execuções deram 6.647,4 e 14.392,5. Essa variação entre sementes é o motivo de a interface usar a semente 42 por padrão: ao comparar parâmetros, a diferença no resultado passa a refletir o parâmetro, e não o acaso.

8.5 Impacto do peso de prioridade

Para medir o efeito do termo de prioridade (hipótese H2), o cenário de 15 cidades foi otimizado com três pesos, cinco execuções cada (configuração B). A métrica de interesse é a distância média percorrida até chegar a cada entrega crítica.

Peso	Distância total média (km)	Km médio até entregas críticas	Execuções viáveis
0,0	2.257,8	203,7	5/5
0,5	2.294,9	178,6	5/5
2,0	2.282,2	198,5	5/5

Com peso 0,5, as entregas críticas passaram a ser alcançadas, em média, após 178,6 km, contra 203,7 km sem o termo de prioridade, uma antecipação de 12,4%, com 1,6% a mais de distância. O efeito é menor que na primeira versão do sistema, quando só distância e prioridade disputavam a ordem das visitas: agora as janelas de horário, com penalidade de 50 por minuto de atraso, e a separação dos grupos por veículo restringem muito a ordem possível. Com peso 2,0 não houve ganho adicional, pelo mesmo motivo. H2 é confirmada com essa ressalva; quando a prioridade precisa ser garantida, o modo hard impede qualquer entrega crítica depois de uma regular no mesmo veículo.

8.6 Operador de reparo

Para avaliar o operador de reparo (hipótese H4), os cenários de 15 e 25 cidades foram executados com e sem o reparo (configuração B, 5 execuções cada).

Cenário	Operador de reparo	Viáveis	Violações sanitárias	Distância média (km)	Fitness médio
15 cidades	com reparo	5/5	0	2.305,7	5.231,9
15 cidades	sem reparo	5/5	0	2.307,7	9.619,5
25 cidades	com reparo	5/5	0	3.918,6	84.821,8
25 cidades	sem reparo	5/5	0	3.915,6	139.105,7

Nos dois cenários, todas as execuções terminaram viáveis mesmo sem o reparo: a semente do vizinho mais próximo já respeita as regras sanitárias e o elitismo a preserva. A diferença está na qualidade: sem o reparo, o fitness médio foi 83,9% maior no cenário de 15 cidades e 64,0% maior no de 25, com distâncias praticamente iguais, ou seja, com mais atrasos. Sem o reparo, a maioria dos filhos gerados pelo crossover e pela mutação viola alguma regra sanitária e é

descartada pela penalidade; com ele, esses filhos viram soluções viáveis e o AG gasta as gerações melhorando a ordem das visitas. H4 é confirmada de outra forma: o reparo não foi necessário para a viabilidade, mas melhorou bastante o resultado.

8.7 Composição da frota

A tabela mostra o cenário de 15 cidades com quatro frotas diferentes (configuração B, 3 execuções cada).

Frota	Veículos	Viáveis	Distância média (km)	Veículos usados	Atraso médio (min)
Padrão (1 de cada tipo)	5	3/3	2.228,6	4,0	178
Sem caminhão sem refrigeração	4	0/3	2.300,8	3,3	400
Sem caminhão com refrigeração	4	0/3	2.278,7	3,7	527
Padrão + 1 caminhão refrigerado	6	3/3	2.104,1	5,7	0

Sem o caminhão refrigerado, a entrega do grupo B não tem veículo apto e nenhuma solução é viável; na aplicação, a pré-verificação detecta isso antes de executar o AG e responde com o motivo. Sem o caminhão comum, a frota restante não consegue separar os grupos A e C respeitando capacidade e autonomia (a moto leva só 15 kg e o carro elétrico, sem recarga, não alcança Bauru), e sobra uma violação de contaminação. Com um caminhão refrigerado a mais, que também pode levar outros grupos quando não carrega o B, a distância caiu 5,6% e os atrasos foram eliminados. O experimento mostra como o sistema apoia decisões de frota: o gestor pode simular a inclusão ou retirada de um veículo antes de mudar a operação.

8.8 Abastecimento em trechos longos

Para validar o abastecimento no caminho, foi simulada uma entrega de Porto Alegre (RS) a Fortaleza (CE), ida e volta, com um carro (autonomia de 500 km, autonomia útil de 425 km com a reserva de 15%). O trecho tem cerca de 4.200 km. A simulação gerou 19 abastecimentos: 9 no caminho em cada sentido, a cada 425 km, e um em Fortaleza antes da volta. Nenhum trecho termina abaixo da reserva, e os 15 minutos de cada parada entram nos horários seguintes. Na mesma rota com distâncias reais do Google (4.197 km na ida), os pontos de

parada foram localizados sobre a geometria da estrada e nomeados por reverse geocoding:

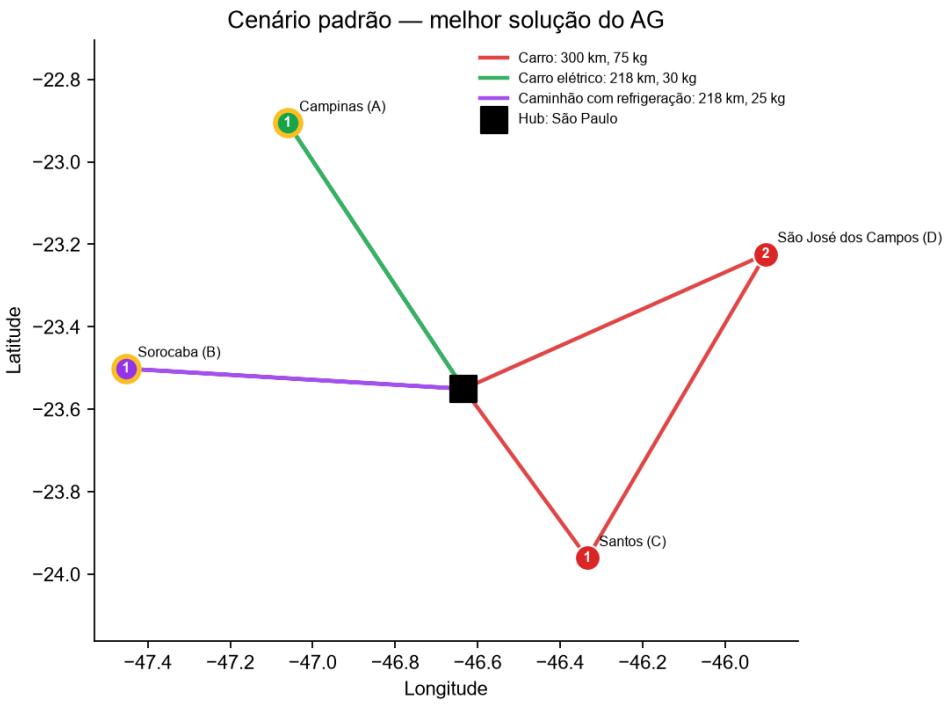
Parada	Km do trecho	Município estimado (Google)
1	425	Palhoça, SC
2	850	Barra do Turvo, SP
3	1.275	Araras, SP
4	1.700	Uberlândia, MG
5	2.125	Brasília, DF
6	2.550	São Desidério, BA
7	2.975	Gilbués, PI
8	3.400	Brejo do Piauí, PI
9	3.825	Tauá, CE

Paradas para abastecer no trecho Porto Alegre → Fortaleza (carro).

A localização das 9 paradas exigiu uma chamada à Routes API para a geometria do trecho e 9 consultas de reverse geocoding, feitas em paralelo. Na primeira versão do sistema, esse mesmo trecho aparecia com um único abastecimento e chegada com 0% da autonomia, o que motivou a mudança.

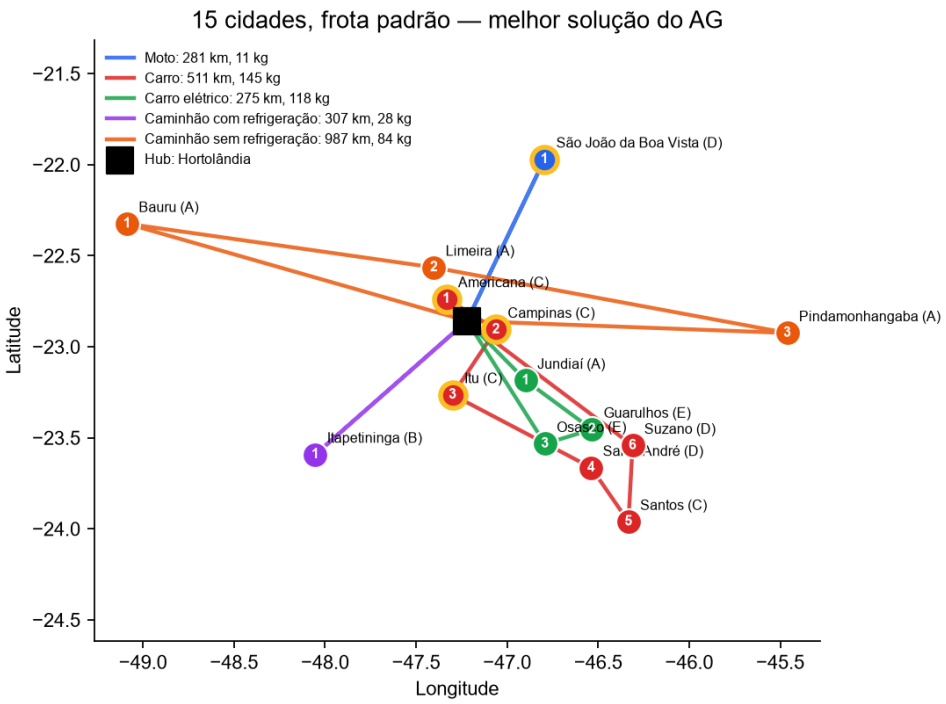
9. Visualizações e análise das rotas

Na aplicação, as rotas são desenhadas sobre um mapa real (Leaflet + OpenStreetMap) com a geometria das estradas obtida da Routes API; cada veículo tem uma cor, as paradas são numeradas na ordem de visita, as entregas críticas têm destaque, o hub tem um marcador próprio e as paradas para abastecer no caminho aparecem com o ícone ⛽. Ao lado do mapa, o painel de rotas por veículo mostra horários de chegada, janelas, esperas, atrasos e abastecimentos. As figuras a seguir reproduzem as melhores soluções dos experimentos em coordenadas geográficas.



Cenário padrão: três veículos, separados pelas regras de transporte (entre parênteses, o grupo de cada entrega; contorno amarelo indica entrega crítica).

No cenário padrão, Campinas (grupo A) e Santos (grupo C) não podem ir no mesmo veículo, e Sorocaba (grupo B) exige o caminhão refrigerado e precisa viajar sozinha. O AG usa o carro para Santos e São José dos Campos (300 km, 75 kg), o carro elétrico para Campinas (218 km, dentro dos 300 km de autonomia) e o caminhão refrigerado para Sorocaba. A moto e o caminhão comum ficam no hub. As quatro entregas chegam dentro das janelas.



Cenário de 15 cidades: cinco veículos, cada um com grupos compatíveis (entre parênteses, o grupo de cada entrega; contorno amarelo indica entrega crítica).

No cenário de 15 cidades, todos os veículos saem. O carro leva só os grupos C e D (511 km, 145 kg) e visita primeiro as entregas críticas (Americana, Campinas e Itu); o carro elétrico leva A e E (275 km, dentro dos 300 km); o caminhão sem refrigeração concentra as entregas do grupo A mais distantes (987 km); o caminhão refrigerado atende sozinho Itapetininga (grupo B); e a moto leva os 11 kg de São João da Boa Vista, abastecendo no caminho. O atraso total é de 18 minutos e a solução tem 5 abastecimentos.

10. Testes automatizados

O backend tem 96 testes automatizados com pytest (81 funções de teste, algumas parametrizadas), que rodam sem acesso à internet: Google, IBGE e OpenRouter são substituídos por respostas simuladas ou pelos fallbacks. Os testes rodam a cada push no workflow de CI, junto com o build de produção do frontend, e o deploy só acontece com a CI verde.

Arquivo	Testes	O que é validado
test_genetic_algorithm.py	11	split/join de rotas, operadores sempre geram permutações válidas, AG nunca pior que o vizinho mais próximo, capacidade força mais veículos, prioridade,

		origem e destino distintos, semente reprodutível e parada por estagnação
test_constraints.py	9	Capacidade, autonomia e prioridade em modo soft e hard; respostas 422 com o motivo quando a restrição rígida é impossível
test_transport.py	21	Contaminação e refrigeração sempre rígidas, autonomia do carro elétrico, janelas (espera e atraso), operador de reparo, pré-verificação da frota, abastecimento na cidade e no caminho (inclusive trecho de 4.000 km), localização do ponto na geometria
test_llm.py	13	Retentativas (429, resposta vazia ou truncada), modelos de reserva, prompts por público, contexto completo, sugestões por regras e histórico de execuções com as mesmas cidades
test_geo.py	11	Estados, municípios do IBGE, validação de nomes com sugestões, geocodificação com e sem Google
test_auth.py	5	Rotas exigem token; login válido e inválido; token expirado ou com outra assinatura é rejeitado
test_api.py	11	Endpoints de saúde e cidades, validação de parâmetros, otimização padrão e com origem/destino, perguntas, semente padrão reprodutível e decodificação de polyline do Google

```
cd backend
pytest -q
96 passed
```

11. Desafios enfrentados e soluções

Desafio	Solução adotada
Representar vários veículos de tipos diferentes sem quebrar os operadores de permutação	Separadores negativos únicos; a posição da rota no cromossomo define o veículo da frota
Regras sanitárias (contaminação e refrigeração) raramente satisfeitas por acaso	Operador de reparo, heurística inicial que respeita as regras e penalidade sempre rígida
Restrições com importâncias diferentes para cada cenário	Modo soft ou hard por restrição, com pré-verificação e resposta 422 explicando o motivo
Trechos maiores que a autonomia (ex.: Porto Alegre → Fortaleza) exibiam um único abastecimento e chegada com 0%	Abastecimento no caminho a cada vez que a reserva é atingida, com o ponto localizado na geometria da rota e nomeado por reverse geocoding em paralelo
APIs de distância do Google em status de legado	Routes API (computeRouteMatrix e computeRoutes)
Limites da Routes API (625 elementos por matriz, 25 waypoints)	Requisições em blocos de 25x25 e rota em trechos
Nome de cidade confundido com distrito homônimo (ex.: "Campinas" no RJ)	Validação na lista oficial de municípios do IBGE, com sugestões de nomes próximos
Escala do fitness com penalidades, que distorce a roleta	Seleção por torneio, que depende só da ordem
Comparar parâmetros com resultados que variam a cada execução	Semente configurável (padrão 42)
Execuções longas sem ganho depois da convergência	Critério de parada por estagnação

Limites de uso (429), respostas vazias ou truncadas da LLM	Retentativas com Retry-After, modelos de reserva, aumento do limite de tokens e esforço de raciocínio baixo
Risco de a LLM inventar dados ou repetir sugestões já testadas	Contexto restrito ao JSON, regras explícitas, temperatura baixa e histórico das execuções com as mesmas cidades
Publicar na nuvem sem expor chaves	Secret Manager e Workload Identity Federation, sem chave de conta de serviço no GitHub
Uso em celulares e telas pequenas	Layout responsivo com tabelas convertidas em cartões
Execução sem chaves de API (testes, demonstrações)	Fallback haversine para distâncias e textos gerados por regras

12. Discussão crítica

Os resultados mostram que o algoritmo genético, adaptado do código base, é uma ferramenta adequada para o roteamento hospitalar com restrições heterogêneas. Quando só a distância importa, uma heurística simples como o vizinho mais próximo chega perto do AG; quando entram janelas de horário, regras sanitárias e frota heterogênea, as restrições interagem e o AG passa a fazer diferença: no cenário de 15 cidades, reduziu o atraso total de 781 para 18 minutos com custo pequeno de distância. A configuração de cada restrição como soft ou hard dá ao gestor o controle do compromisso: aceitar um pequeno atraso para economizar quilômetros, ou exigir o cumprimento rígido e receber uma explicação quando ele for impossível. A LLM transforma o resultado numérico em material operacional para os motoristas e em diagnóstico para a equipe técnica, e o histórico de execuções evita que ela sugira valores que já foram testados.

Algumas limitações merecem atenção. O AG não garante o ótimo global, e seu custo cresce com população × gerações × número de pontos: a configuração de melhor qualidade levou dezenas de segundos nos cenários maiores. O critério de estagnação reduz esse custo, mas, com muitas janelas de horário, interrompe a busca em patamares que seriam superados mais adiante. Uma busca local (2-opt ou realocação) aplicada ao melhor indivíduo, formando um algoritmo memético, e a comparação com solvers especializados como o OR-Tools seriam passos naturais. O tratamento das janelas como soft, com penalidade linear por minuto, também faz o resultado depender da escala da penalidade em relação à distância, o que o usuário precisa calibrar.

Do lado dos dados, as demandas, prioridades, grupos e janelas das cidades sorteadas são sintéticos, e os experimentos usaram distâncias estimadas para garantir reprodutibilidade; com o Google, as distâncias e os tempos reais tendem a ser maiores, mas o algoritmo trabalha com a matriz que receber. Os resultados ficam em memória no Cloud Run, que escala até zero: o histórico de execuções usado pela análise técnica se perde quando a instância é desligada. Por fim, as respostas da LLM devem ser tratadas como apoio: a rota e os números vêm do algoritmo, e a decisão final sobre a operação continua com a equipe de logística.

13. Conclusão

O objetivo de construir um sistema de otimização de rotas para a distribuição de medicamentos e insumos foi alcançado com uma solução completa e publicada: o código base de TSP foi estendido para um VRP com frota heterogênea, prioridades, capacidade, autonomia com abastecimento, janelas de horário e regras sanitárias de transporte, cada uma configurável como soft ou hard; as localizações e distâncias vêm do Google Maps, com validação dos municípios no IBGE; a solução é exposta por uma API autenticada e uma interface web responsiva com mapa, publicadas no Google Cloud com CI/CD; e uma LLM gera a análise técnica, comparando as execuções anteriores, e as instruções dos motoristas, além de responder perguntas em linguagem natural.

Nos experimentos, o AG reduziu o fitness médio em 91,1% e 74,6% em relação ao vizinho mais próximo nos cenários de 15 e 25 cidades, principalmente pela redução dos atrasos, sempre com todas as restrições rígidas atendidas. As hipóteses H1 e H3 foram confirmadas; H2, H4 e H5 foram confirmadas com as ressalvas discutidas: a prioridade tem efeito menor quando disputa com as janelas, o reparo melhora a qualidade mais do que a viabilidade, e a parada por estagnação economiza tempo com alguma perda de qualidade em cenários difíceis.

Como próximos passos, destacam-se: busca local híbrida (algoritmo memético); trânsito em tempo real (Routes API com TRAFFIC_AWARE); múltiplas viagens por veículo e múltiplos depósitos; persistência dos resultados e do histórico em banco de dados, para relatórios semanais e para manter o contexto da LLM entre reinícios; e avaliação sistemática de diferentes LLMs do OpenRouter com os

critérios da seção 7.7. Essa base também prepara a integração com o assistente médico previsto para o módulo 3.

14. Referências

FIAP. genetic_algorithm_tsp: código base de algoritmo genético para o problema do caixeiro viajante. GitHub. Disponível em: https://github.com/FIAP/genetic_algorithm_tsp.

HOLLAND, J. H. Adaptation in Natural and Artificial Systems. Ann Arbor: University of Michigan Press, 1975.

GOLDBERG, D. E. Genetic Algorithms in Search, Optimization, and Machine Learning. Reading: Addison-Wesley, 1989.

DANTZIG, G. B.; RAMSER, J. H. The Truck Dispatching Problem. Management Science, v. 6, n. 1, p. 80–91, 1959.

SOLOMON, M. M. Algorithms for the Vehicle Routing and Scheduling Problems with Time Window Constraints. Operations Research, v. 35, n. 2, p. 254–265, 1987.

DAVIS, L. Applying Adaptive Algorithms to Epistatic Domains. Proceedings of the 9th International Joint Conference on Artificial Intelligence (IJCAI), p. 162–164, 1985.

TOTH, P.; VIGO, D. (ed.). Vehicle Routing: Problems, Methods, and Applications. 2. ed. Philadelphia: SIAM, 2014.

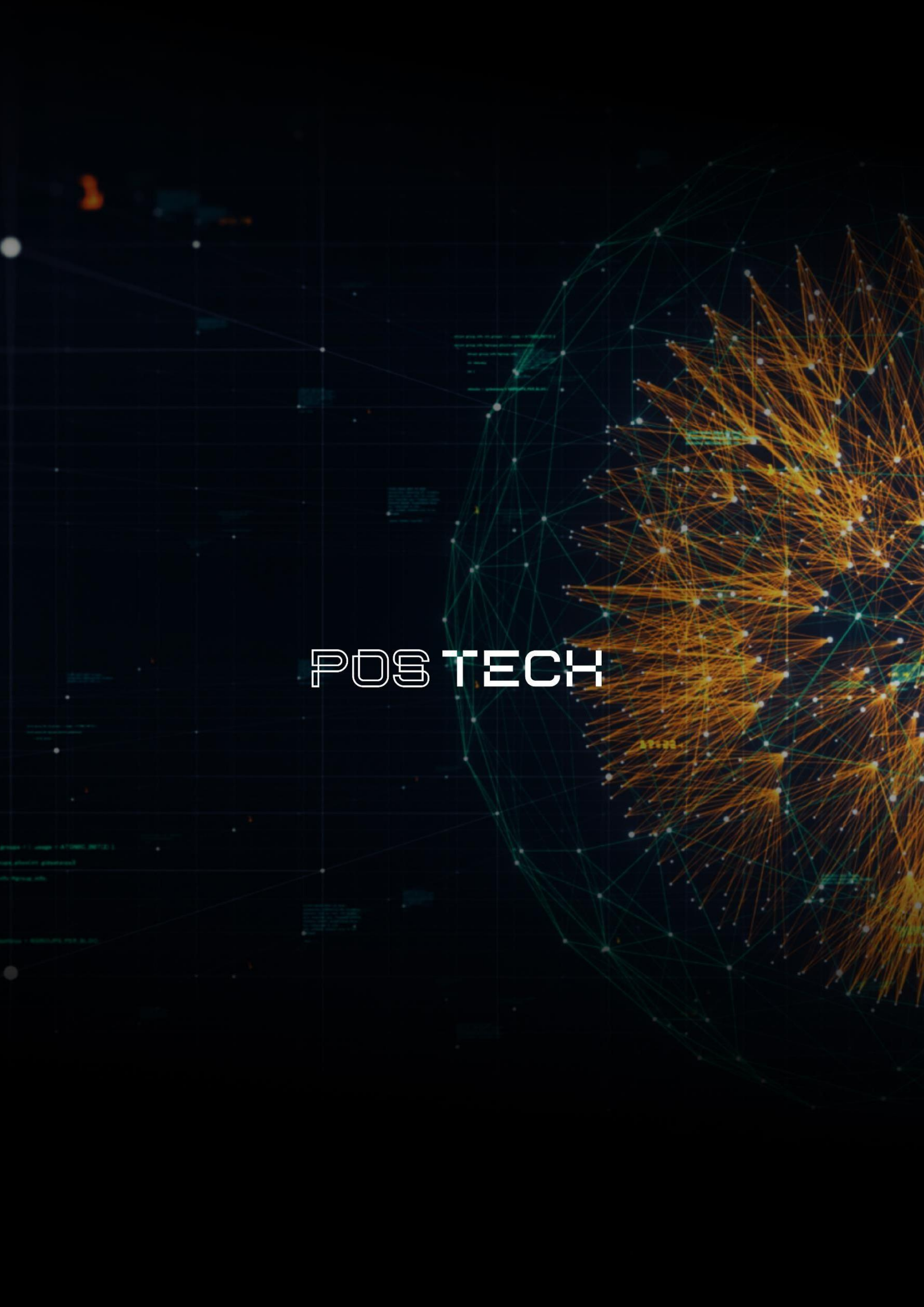
GOOGLE. Routes API e Geocoding API — documentação. Disponível em: https://developers.google.com/maps/documentation.

GOOGLE CLOUD. Cloud Run, Artifact Registry, Secret Manager e Workload Identity Federation — documentação. Disponível em: https://cloud.google.com/docs.

IBGE. API de localidades. Disponível em: https://servicodados.ibge.gov.br/api/docs/localidades.

OPENROUTER. API Reference. Disponível em: https://openrouter.ai/docs.

Documentação das tecnologias utilizadas: FastAPI (fastapi.tiangolo.com), Pydantic (docs.pydantic.dev), PyJWT (pyjwt.readthedocs.io), pytest (docs.pytest.org), Angular e Angular Material (angular.dev, material.angular.dev), Firebase Hosting (firebase.google.com/docs/hosting), Leaflet (leafletjs.com) e OpenStreetMap (openstreetmap.org).


POS TECH